

Introduction to Pandas Basics

Instructor

Prashant Sahu

Manager (Data Science), Analytics Vidhya



What is Pandas?

- 1 Pandas: An open-source data manipulation library.
- 2 Built on NumPy; integrates seamlessly with other data science libraries.
- 3 Used for handling structured data: spreadsheets, SQL tables, and CSV files.



Pandas

Understanding Pandas Data Structures

- Series: A one-dimensional array with labels (similar to a list with indices).
- DataFrame: A two-dimensional labeled data structure (like an Excel table).
- Indexing: helps navigate through these data structures.



Creating and Importing Data in Pandas

- Create DataFrames from dictionaries, lists, or NumPy arrays.
- Import data from CSV, Excel, SQL, and more using `pd.read_csv()`, `pd.read_excel()`, and `pd.read_sql()`.
- Use `.head()` to preview data and `.info()` for structure details.



Exploring DataFrames

- Use `.describe()` for a statistical summary..
- `.shape` and `.columns` provide insights into data structure.
- `dtypes` to understand data types for each column.



Selecting and Filtering Data

1 Select columns with `df['column_name']`.

2 Filter rows using boolean conditions (`df[df['score'] > 50]`).

3 Use `.loc[]` for label-based access and `.iloc[]` for integer-location based access.

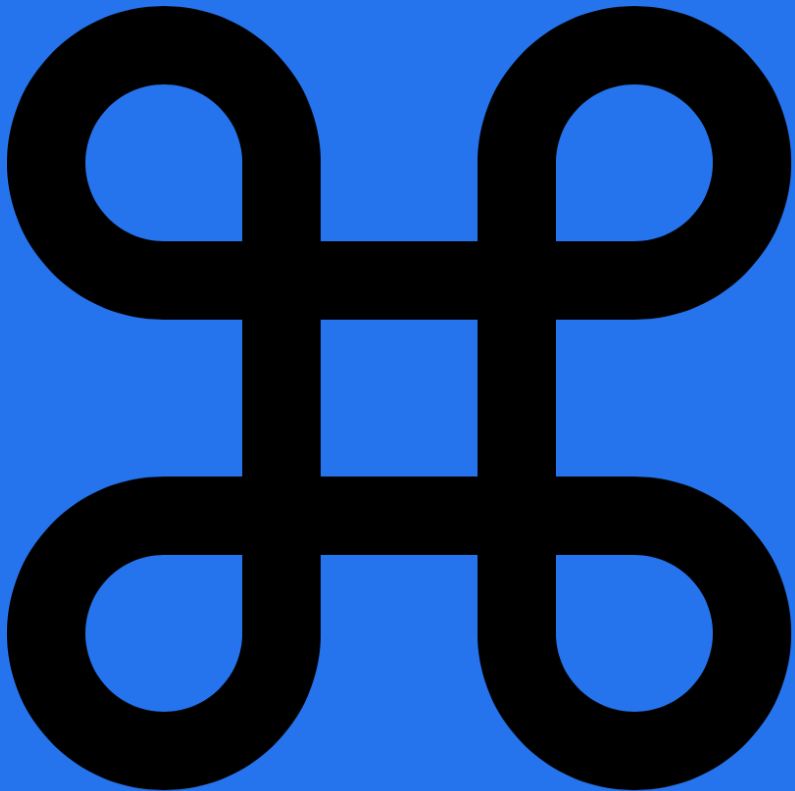


Adding and Modifying Columns Content

- Add new columns by defining values (`df['new_col'] = df['old_col'] * 2`).
- Modify existing columns with `.apply()` for transformations.
- Use `.rename()` to rename columns or rows.



Indexing and Slicing DataFrames



Indexing

helps in labeling data for easy access.

Slicing

allows extracting subsets of data (`df[2:5]`).

Set custom indices with `.set_index()` for better alignment.

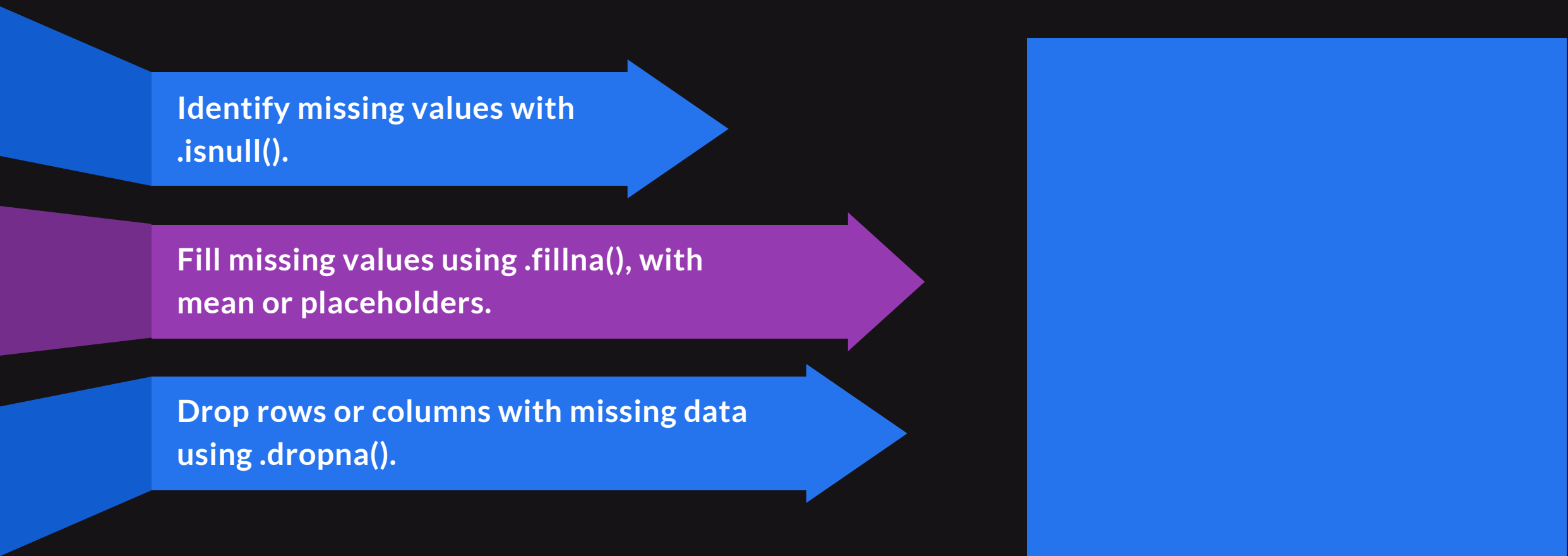
Sorting and Ordering Data

Sort rows with
`.sort_values(by='column_name')`.

Sort by multiple columns for complex ordering.

Use `.sort_index()` to sort by DataFrame
index.

Handling Missing Data



Identify missing values with `.isnull()`.

The diagram illustrates a workflow for handling missing data. It consists of three horizontal arrows pointing from left to right, each representing a step. The first arrow is blue and contains the text 'Identify missing values with .isnull()'. The second arrow is purple and contains the text 'Fill missing values using .fillna(), with mean or placeholders.'. The third arrow is blue and contains the text 'Drop rows or columns with missing data using .dropna()'. To the right of these arrows is a large, solid blue rectangle, which likely represents the final state of the data after these steps are completed.

Fill missing values using `.fillna()`, with mean or placeholders.

Drop rows or columns with missing data using `.dropna()`.

Data Cleaning Technique

- Use `.drop_duplicates()` to remove duplicate rows.
- Convert data types using `.astype()` to ensure data integrity.
- Standardize data with `.str.lower()` for text fields.



Grouping Data with GroupBy

- Use `.groupby()` to group data by one or more keys.
- Aggregate using `.sum()`, `.mean()`, `.count()`.
- Example: Group sales data by region to calculate regional sales totals.



Merging and Joining DataFrames

- Merge DataFrames using `.merge()`; specify type (`inner`, `outer`, `left`, `right`).
- Use `.concat()` to add rows or columns.
- Real-world example: Merge customer profiles with sales data.



Pivot Tables in Pandas

- Create pivot tables with `.pivot_table()`.
- Summarize data by one or more dimensions.
- Example: Pivot table for summarizing sales by region and product.



Applying Functions Using `.apply()` and `.map()`

- Use `.apply()` to apply a function to a column or row.
- `.map()` is great for transforming values in a Series.
- Real-world application: Apply discounts or categorize values.



Handling Dates and Times in Pandas

- Convert strings to dates using `pd.to_datetime()`
- Extract components like year, month, or day.
- Useful for time-series analysis and trend detection.



Aggregation Functions

- Convert strings to dates using `pd.to_datetime()`
- Extract components like year, month, or day.
- Useful for time-series analysis and trend detection.



String Operations in Python

- **.str accessor**: Apply string functions like **.lower()**, **.contains()**, **.replace()**.
- Useful for text cleaning, formatting, and searching.
- Real-world scenario: Clean and standardize customer name formats.



Visualization with Pandas

- Use `.plot()` to create quick visualizations.
- Create line, bar, and scatter plots to explore data.
- Integrate with Matplotlib for advanced plotting.



Thank You
